PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination

Yateen Vaviya
Department of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, Maharashtra, India
yateen.vaviya23@spit.ac.in

Vinayak Yadav
Department of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, Maharashtra, India
vinayak.yadav23@spit.ac.in

Sahil Gupta
Department of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, Maharashtra, India
sahil.gupta23@spit.ac.in

Dr. Rupali Sawant
Department of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, Maharashtra, India
rupali_sawant@spit.ac.in

Prof. Dayanand Ambawade
Department of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, Maharashtra, India
dd_ambawade@spit.ac.in

***Abstract*—Distributed computing frameworks presuppose a configured cluster, a program written against their API, and machines that can reach one another. PipedPeer removes all three for Python: one command runs an unmodified script on other machines, each running the same daemon, with no head node and no scheduler process. The script's environment is built as a Nix closure against a pinned package set, so it has the same store path on every machine, and a peer receives only the store paths it lacks. The parallel primitives the script already uses are substituted at import time; parallel maps are divided by a makespan solver over measured peer throughput, and numerical work moves only when a cost model predicts a gain. Machines behind different routers meet through an introducer that any member can serve, then connect directly over authenticated QUIC; no data is relayed. In a container lab a repeat submission took 0.83 s against 23.4 s for the first, and interception cost $1.24\times$ the start-up of a trivial job. Between two laptops in different homes, over a Tailscale network, a CPU-bound script ran in 13.3–14.6 s against 17.4–18.1 s on one.**

***Index Terms*—distributed computing, peer-to-peer, NAT traversal, transparent interception, reproducible environments, content-addressed caching, cost-based scheduling**

I. Introduction

The gap between running `python script.py` and running the same work on several machines is large in operational effort and often small in the computation involved. Apache Spark [1], Ray [2] and Dask [3] presuppose a configured cluster with a head node, matching software on every worker, a scheduler process, and a network on which every machine reaches every other. Remote execution over `ssh` avoids the set-up but provides no environment management, placement, isolation or fault tolerance, and still needs a reachable address.

Three obstacles are usually treated separately. The *environment* obstacle: a remote machine must be made to match the local one. The *code* obstacle: using more than one machine normally means rewriting the program. The *reachability* obstacle: machines behind different home or campus routers accept no inbound connections, and the usual answers — a VPN, port forwarding or a relay — are either configuration or a bottleneck. PipedPeer addresses all three. Its contributions are:

1) **Peer-to-peer closure distribution.** Building on Nix's content-addressed store paths [4], we push signed closures between peers, send only the store paths a receiver lacks, and offer distributed work only to peers that can run the closure (Section III-B).
2) **Transparent interception with measured work sharing.** Parallel primitives are substituted at import time, parallel maps are divided by a makespan solver over measured rates, and numerical work is gated by a cost model (Sections III-D, III-E).
3) **Joining across networks without a relay**, through an introducer that only exchanges addresses and that every daemon serves (Section III-C).
4) **Placement without a scheduler**: filters and a bounded score evaluated in the submitting client, with a three-phase lease (Section III-A).
5) **A fallback that preserves the result on every distributed path** (Section III-F).

II. Related Work

Spark [1], Ray [2] and Dask [3] are mature and far more capable at scale. Ray ships a drop-in `multiprocessing.Pool` and a `joblib` backend, and Modin [5] provides distributed pandas behind the pandas API; each needs a changed import and a running cluster. PipedPeer changes no line of the program, because substitution happens before the first user statement, and it decides per call whether to distribute. BOINC [6] pioneered

TABLE I
COMPARISON WITH EXISTING SYSTEMS. THE LOWER BLOCK IS WHERE OURS IS THE WEAKER SYSTEM.

Property	Ours	Spark	Ray	Dask	ssh
Runs unmodified Python	yes	no	part.	part.	yes
No cluster to configure	yes	no	no	no	part.
No head or master node	yes	no	no	no	yes
Reproduces the environment	yes	part.	part.	part.	no
Joins NATed machines, no relay	yes	no	no	no	no
Sandboxes each job	part.	no	no	no	no
Survives losing a node	yes	yes	yes	yes	no
Authenticated, encrypted	part.	yes	part.	part.	yes
Runtimes beyond Python	**no**	yes	part.	no	yes
Proven in production	**no**	yes	yes	yes	yes

volunteer computing, but as a platform for specific projects rather than a general executor. Table I summarises the differences.

The cross-network mechanism follows established practice. Reflexive addresses are discovered as in STUN [7]; port mappings are requested with PCP [8] and NAT-PMP [9]; simultaneous UDP probing from the socket that will carry the connection is the hole punching of Ford et al. [10]; links are QUIC [11].

Edge-computing offloading studies the decision our cost model makes. Deng et al. [12] weigh transmission against local computation; Yong et al. [13] show partial offloading beating all-or-nothing, the regime our interception works in; learned policies [14] exceed a fixed rule at the price of training; and topology-aware placement [15] outperforms scalar scoring, a limitation our placement shares. Spenger [16] defines failure transparency, under which application code never observes a failure; Section III-F must provide it for scripts that contain no failure handling. For training we build on PyTorch DDP [24], 8-bit gradient compression [26] with error feedback [27], and periodic averaging as in DiLoCo [29] and local SGD [30].

III. System Design

Design goals. (G1) The script runs unchanged. (G2) Joining needs one set-up command, plus one code across networks. (G3) No machine is special: there is no head node, scheduler or registry, and every machine both submits and works. (G4) Distribution never changes the result and is declined when it would be slower. (G5) No job data passes through a third machine.

Coordination is *decentralised*: every daemon serves the introducer for its cluster, placement is computed in the submitting client, and a training ring's lead is simply the node that runs rank 0 of that job. It is *recursive* in that work is divided again wherever it lands: a node that cannot hold a parallel map forwards it whole to a peer, that peer splits it among its own peers, and each part spreads across the cores of the node that runs it.

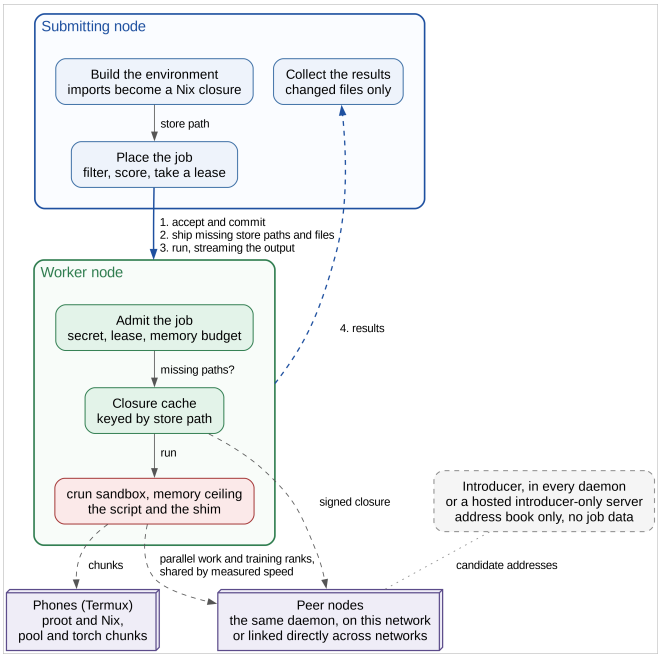


Fig. 1. System design. Both roles are the same binary; peers on other networks are reached over direct links that the introducer helps establish but never carries.

Every machine runs one daemon; the command-line client is short-lived. The coordinator is a library linked into the client, so placement is decided where the job is submitted (Fig. 1). On one network participants find each other unaided; across networks a participant runs `pipedpeer connect` with a one-line code that another member's `pipedpeer invite` printed, carrying the cluster secret and a few introducer addresses.

A. Placement

Candidates come from known peers, LAN discovery and internet-linked peers. Hard filters run in order — architecture, GPU if required, then free memory, cores and VRAM — and survivors are scored:

$$S = \Big(1 - \frac{u_{\text{cpu}} + u_{\text{mem}}}{200} - 0.05\, n_{\text{jobs}} + 0.10(\hat{c} - 0.5) + 0.06(\hat{f} - 0.5) + 0.10(\hat{m} - 0.5) + 0.10(\hat{r} - 0.5)\Big) \cdot h \tag{1}$$

where u_{cpu} and u_{mem} are processor and memory utilisation in percent, n_{jobs} the running jobs, $\hat{c}, \hat{f}, \hat{m}, \hat{r}$ core count, clock, free memory and free-core ratio normalised onto $[0, 1]$, and $h \in [0, 1]$ the node's health score. Load dominates hardware: the load terms can subtract 1.0, the hardware terms move the score by at most 0.36, and a missing metric takes the neutral 0.5. Free memory is what a node can *grant*: where a user `systemd` is available, each daemon runs under a cgroup ceiling of half the machine, and co-located daemons (several on one machine) subtract one another's reservations.

Work is reserved by a three-phase lease; the concurrency and memory checks and the insertion happen under one lock. The submitter renews every 30 s, also while the environment builds and uploads, against a five-minute expiry. A submitter that dies stops renewing and its slot is reclaimed; a worker that dies leaves the healthy set and its job is rescheduled with back-off.

B. Environment Reproduction

The client scans imports, resolves them to packages, and builds a Nix closure from a flake pinned to an *exact* package-set revision, so the store path is a stable name for an environment across machines and time. Imports absent from the mapping table are looked up by name in the pinned set, the scan follows the script's own modules, a `requirements.txt` is honoured, and an import still unresolved is reported before the run.

Before uploading, the client asks whether the target can already run the closure; if not, it sends only the store paths the target lacks. After an upload the worker pushes the closure to peers that cannot yet run it, because only those that can are offered distributed work. Between daemons the push is a binary cache produced and imported by Nix itself and signed with the sending node's key; the receiver checks that key against the signer's published identity before trusting it for that import. Closures go only to peers of the same architecture; a peer of another architecture receives the package list and builds its own environment once. A phone joins under Termux [17], where `proot-distro` provides, through `proot` [18], a Debian root with single-user Nix, and takes parallel chunks only.

C. Joining Across Networks

An introducer keeps an address book, keyed by a hash of the cluster secret, and forwards punch requests between two nodes; it carries no job data. Every daemon serves one for its own cluster and registers with every introducer it knows, so no single machine's address is a point of failure. Each node offers its LAN addresses, a port mapped on its router (published only when the router's external address matches the one the introducer observed) and its reflexive address. Probes, introducer messages and QUIC share one UDP socket, because a router's mapping belongs to the socket that opened it; both sides probe every candidate pair every 250 ms for 15 s. When one side's NAT allocates a new port per destination, that side opens 256 sockets toward the peer while the other sends 600 probes to random ports, and the socket that is hit is kept [19].

The winning path carries QUIC with a hello signed by each node's Ed25519 [20] key and checked against the node it meant to reach; the hello signs the session's exported keying material [21] and proves the cluster secret by an HMAC [22] over it, so it cannot be replayed on another session. The link appears locally as a loopback forwarder recorded as one path to that node, so placement, transfer and gradient exchange run over it unchanged. A node is identified by its key's fingerprint, never by its address: the node store keeps every path to it and switches only to a path that answered and proved the key, and QUIC connections migrate when an address changes, so a machine that changes network rejoins on its own. A peer with no path is reported with its reason and routed around, never relayed. The daemon API requires the cluster secret on every route and serves pinned TLS [23] and HTTP on one port.

D. Transparent Interception

A Python package embedded in the binary is placed on the interpreter's path so that it is imported before the user's first line; libraries are patched on first import. Substituted primitives are parallel maps, `joblib`, pandas grouping, joining and chunked reading, NumPy matrix products and decompositions, and data-parallel training.

Parallel maps. A kernel travels as *source* when it is a module-level function, including one in `__main__`, or *by value* through cloudpickle [25] when it is a lambda or closure. The node that holds a map splits it once among its peers, and each peer runs its part on its own cores. The split is a makespan solver: each device is modelled as a set-up time plus a rate, fitted from the parts it has returned, and a device whose set-up alone exceeds the makespan is declined. A part unfinished past its prediction counts as taking at least that long, a part waiting or overdue is copied to a free peer, and the first copy to finish wins.

Dataframes. Grouping and joining hash rows by key, so each key is complete on one node and any aggregation is exact with no combiner; the row order of joins is not preserved.

Training. Ranks post gradients to the lead node's daemon rather than building a rank-to-rank mesh, and the daemon returns one sample-weighted average $\sum_i n_i g_i / \sum_i n_i$, sent as 8-bit values with per-block scales and error feedback. The global batch is partitioned across ranks in proportion to their speed, preserving the effective learning rate, and after a warm-up the lead compares the split's step time with the fastest rank alone and keeps the ring only if it is faster. Samples are assigned to ranks by a weighted rendezvous hash [28]: when a rank dies the survivors take its share at the next epoch boundary while over 95% of their samples stay in place, and a rank that returns joins from the lead's model and optimiser snapshot, taking about $1/n$ of the samples.

A fast mode synchronises every H steps: an exact warm-up, then local steps whose weights are averaged by samples, with optimiser moments averaged every tenth round. It never changes epochs, batch or learning rate, and must keep validation loss at most 1% above one device's (mean of three seeds, every test model). DiLoCo's outer Nesterov step (lr 0.7, momentum 0.9) settles at $0.7/(1-0.9) = 7$ times the averaged change, which suits syncs every few hundred steps; at our intervals of 2–64 steps PipedPeer uses plain averaging (outer lr 1, no momentum), chooses H by ring size (Section IV-D), and uses no interval past four ranks. The lead switches a ring to fast mode once, when the predicted fast step with a 10% margin beats both the exact split and one GPU.

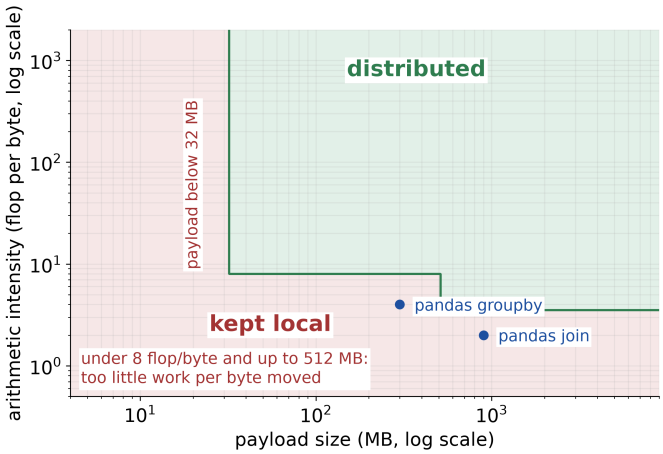


Fig. 2. Decision regions of Equation 2 for the pandas path ($\rho = 1$, 1 GFLOP/s) at three nodes on a 500 MB/s link.

E. The Cost Model

Numerical work is gated. With b payload bytes, ϕ operations per byte, ρ link crossings, B bandwidth, R and R' local and remote throughput and K nodes, divisible work moves only when

$$\frac{b\phi}{R} > \frac{\rho\, b}{B} + \frac{b\phi}{R'K} \times 1.3. \tag{2}$$

Work needs at least two nodes and 32 MB; work that does not fit in local memory moves regardless, and a matrix product that fits nowhere raises an error rather than exhausting the machine. A GPU peer raises R' sixfold. The inequality is first tried at an optimistic 10 Gbit/s and, only if offloading could win, bandwidth is measured with a 64 MB probe through the dispatch path (Fig. 2). Indivisible work (SVD) moves when transfer costs under half the local time. At about 200 GFLOP/s, local BLAS outruns a CPU cluster's transport, so dense matrix products stay local, while a GPU peer can win.

F. Degradation Guarantee and Operation

Using the cluster must never change the result. Parallel-map parts fill a result table first-wins, idle local cores re-run any part not yet returned, and the call returns once every item is covered, so an unresponsive cluster degrades to a local pool. Every intercepted call falls back to the original on exception, a failed peer is skipped, a dead worker is respawned, a failed job is rescheduled with back-off, and a training ring continues on the ranks that are left.

A run behaves like a local one. An interrupt sends one cancel frame; the executing daemon signals the job's process group, and parallel parts stop on every peer because each carries its request's context. A dropped connection never kills the job: the daemon logs every output frame, keeps the lease while the job runs, and lets the submitter reattach and replay the log without gaps or repeats. Each run ends with one line splitting its time into the script's and PipedPeer's, and a summary of where its items ran.

G. Implementation

PipedPeer is about 48,000 lines of Go 1.25 in one binary that is both client and daemon, with about 37,000 lines of tests, and an 8,000-line Python interception package embedded in it. The daemon serves an HTTP API, makes direct links with quic-go, builds environments with Nix flakes, and runs each job in a `crun` sandbox. The suite of 829 tests is checked by a mutation audit that disables 95 features one at a time — among them gradient weighting, co-located daemons' memory accounting and the router-mapping checks — and fails for every one.

IV. Evaluation

We ask four questions. **Q1:** What does interception cost, including when it declines? **Q2:** How much do the environment cache and the closure diff save? **Q3:** Does work cross machines with correct results, and does it pay between physical machines? **Q4:** Does split training keep accuracy, and when is it faster?

A. Setup

A 12-thread laptop runs a lab of three worker daemons in containers, each with a fixed processor and memory limit; where the network matters, links are shaped with `tc netem` to the latency, bandwidth and loss of a LAN, a Wi-Fi network or a 5G link. Because the containers share one processor, the lab measures correctness, overhead, caching and cost growth, and speedup only as one container against several of the same size. A 16-thread laptop with one GPU runs a second lab. Physical runs use two laptops in different homes (RTX 3060 and RTX 3050), a 20-core and a 16-core host on one LAN, and a Samsung Galaxy S20 FE (Android 13). Between the two laptops, PipedPeer's own QUIC link ran on top of a Tailscale (WireGuard) network with 6–8 ms round trip, because the second laptop is shared into the first one's tailnet; these runs measure distribution between homes, not PipedPeer's NAT traversal. NAT traversal without Tailscale is evaluated on a two-router testbed: two separate NATs whose LANs both use 192.168.0.0/24, across which every user command is run. Training accuracy is measured in-process, with several ranks side by side on one machine, against a single device. Lab wall times are medians of three runs unless stated; accuracy is the mean of three seeds.

B. Microbenchmarks (Q1, Q2)

Interception. Start-up of a job that imports nothing heavy takes 0.117 s with interception against 0.094 s without (1.24×, best of five runs), within the project's 3× budget. On a whole job that the cost model declines, a 16-item map with 0.01 s of work, the median over five runs was 0.796 s against 0.678 s, every receipt showing nothing dispatched: declining costs about 0.12 s per job.

Environment cache. The first submission of a job took 23.4 s. The second landed on a worker that had not yet received the closure and took 8.1 s; the next three found it in place, shipped only the workspace, and took a median of 0.83 s

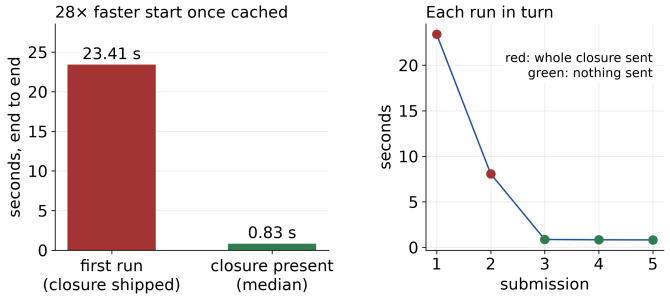


Fig. 3. Five submissions of the same environment in the container lab.

(Fig. 3). Adding one package to an environment a peer already held sent 7 of 43 store paths, 28.6 MB instead of 206.0 MB.

Work split. On a lab limited to 8, 2 and 1 CPUs, a headcount split was 1.6× slower than not distributing; the measured split declined the 1-CPU worker and took 2.87 s against 3.61 s (medians of five). With one peer at a tenth of the CPU, four consecutive runs took 1.60, 1.25, 1.09 and 1.08× an equal cluster's time as the rate estimates converged.

Introducer. During a run between peers the introducer's host carried 0.2 MB against about 30 MB between the peers, and with the introducer suspended a known peer was reached in 0.03 s.

C. End-to-end runs (Q3)

With distribution forced in the three-worker lab, 18 of 48 parallel-map items ran on three other daemons in each of five runs, with identical checksums. In the second lab a parallel map took 43.5 s on its first run and 17.0 s with its environment cached; a GPU job submitted from a node without a GPU ran on the GPU node in 7.6 s cached (264.8 s the first time); an interrupt stopped work on every peer within 2 s; after the introducer changed address, a three-node cluster reconnected with no command in 12.1–12.6 s over three runs; a detached job was reattached and returned its results; and a two-rank ring finished its 80 epochs after one rank was killed, in 90.5 s.

Table II lists the physical-machine results. A parallel map and a mixed CPU script (pool, joblib, NumPy, pandas) ran faster on two machines than on one, the latter with about half its items on the second laptop. The first run of a new GPU environment is dominated by its build (487 s of 506 s, mostly downloading CUDA PyTorch); the same script then took 17.7 s. On the phone, a laptop's CPU script spread over both laptops and the phone gave the phone 8 of 192 items with an unchanged checksum, after the phone had built the environment from the cache in 27 s; Nix's Python inside `proot` ran it in 62–66 s against 73–81 s for Termux's own Python (three runs each).

D. Training (Q4)

When splitting pays. In the lab, a small model with heavy steps split over two containers beat one (9.1 against 11.8 s), while a 9 MB model with light steps over Wi-Fi ran ten times slower split than alone (80.7 against 8.0 s). Between physical machines neither split beat one GPU (Table II), which the system reported at the end of the run; the lead keeps a ring only when it measures a net speedup. With automatic sync, the two-laptop ring ran 19 exact steps, switched to fast mode and finished in 14.6 s with 25 syncs: auto pays a short exact warm-up before it switches. Measured batch sizes beat equal ones on an uneven ring.

TABLE II
RESULTS BETWEEN PHYSICAL MACHINES

Measurement	Result
Pool map, 20-core host alone vs. with 16-core peer (LAN)	5.8–6.4 vs. 4.9–5.4 s
CPU script, one laptop vs. two over Tailscale	17.4–18.1 vs. 13.3–14.6 s
Training at 54 Mbit/s: 1 machine / 2 ranks fp16 / int8	54.9 / 69.4 / 56.1 s
final loss, same three	0.087495 / 0.087497 / 0.087688
Training, two laptops (Tailscale): 1 GPU / exact / fast / auto	4.2 / 37.1 / 8.5 / 14.6 s
validation loss, same four	0.0794 / 0.0797 / 0.0797 / 0.0796
Ring of 16/8/2 cores, equal vs. measured batches	25.4 vs. 23.6 s
New GPU environment, first run vs. repeat	506 vs. 17.7 s

Fast synchronisation. On two ranks with LAN and Wi-Fi profiles, plain averaging every 64 steps after a 10-step warm-up changed validation loss against one device by $+0.64/+0.61\%$ on a first perceptron, $-2.14/-2.09\%$ on a second and $+0.18/+0.50\%$ on a small transformer (each pair LAN/Wi-Fi), each within exact sync's own spread, while sending 9–15× fewer bytes per rank; it finished 1.2–1.4× faster than exact sync on LAN and 1.6–3.1× on Wi-Fi (one seed). With DiLoCo's outer step the transformer was $+21.7\%$ even at 156 local steps.

Many ranks. The largest interval passing on every model was 32 at two ranks, 16 at three and 4 at four; at eight ranks the transformer failed at every interval down to 2, so no interval is used for five or more ranks. At 4, 8 and 11 ranks, only exact synchronisation with rank-16 PowerSGD [31] compression and error feedback kept every model within 1% of one device (worst mean $+0.18\%$), sending 3.2× fewer bytes than int8; linear [32], square-root [33] and AdaScale [34] learning-rate scaling, doubled epochs, SparseLoCo [35], top-k [36], FedProx [37] and a larger outer learning rate each failed.

V. Limitations

The labs share one processor among their workers, so they show cost, not speedup, and the physical runs cover two or three machines with few repetitions. TLS is trust on first use, the keys trusted to sign closures are not yet bound to an authenticated identity, and authorisation is one shared secret. Only Python is supported, and a training ring depends on rank 0's machine.

VI. Conclusion and Future Work

PipedPeer runs unmodified Python on other machines without configuring a cluster, by reproducing environments as content-addressed closures that are cached and diffed by store path, substituting the parallel primitives a script already uses, dividing work by measured speed, and joining machines across networks over direct, authenticated links with no relay and no privileged node. Its central design rule is to measure before distributing and to decline when distributing would be slower: the per-call cost gate, the measured split and the training ring's own check each follow from it. Future work is to bind signing keys to authenticated identities and require TLS, to add locality- and topology-aware placement and AMD accelerators, to let a training ring survive its submitter and rank 0, and to run a controlled evaluation on more machines.

References

[1] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in *Proc. 2nd USENIX Workshop on Hot Topics in Cloud Computing*, 2010.

[2] P. Moritz et al., "Ray: A distributed framework for emerging AI applications," in *Proc. 13th USENIX OSDI*, 2018, pp. 561–577.

[3] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," in *Proc. 14th Python in Science Conf.*, 2015, pp. 126–132.

[4] E. Dolstra, M. de Jonge, and E. Visser, "Nix: A safe and policy-free system for software deployment," in *Proc. 18th LISA*, 2004, pp. 79–92.

[5] D. Petersohn et al., "Towards scalable dataframe systems," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 2033–2046, 2020.

[6] D. P. Anderson, "BOINC: A system for public-resource computing and storage," in *Proc. 5th IEEE/ACM Workshop on Grid Computing*, 2004, pp. 4–10.

[7] M. Petit-Huguenin, G. Salgueiro, J. Rosenberg, D. Wing, R. Mahy, and P. Matthews, "Session Traversal Utilities for NAT (STUN)," RFC 8489, IETF, Feb. 2020.

[8] D. Wing, S. Cheshire, M. Boucadair, R. Penno, and P. Selkirk, "Port Control Protocol (PCP)," RFC 6887, IETF, Apr. 2013.

[9] S. Cheshire and M. Krochmal, "NAT Port Mapping Protocol (NAT-PMP)," RFC 6886, IETF, Apr. 2013.

[10] B. Ford, P. Srisuresh, and D. Kegel, "Peer-to-peer communication across network address translators," in *Proc. USENIX Annual Technical Conf.*, 2005, pp. 179–192.

[11] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, IETF, May 2021.

[12] Y. Deng, Z. Chen, X. Chen, and Y. Fang, "Task offloading in multi-hop relay-aided multi-access edge computing," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 1372–1376, 2023.

[13] D. Yong, R. Liu, X. Jia, and Y. Gu, "Joint optimization of multi-user partial offloading strategy and resource allocation strategy in D2D-enabled MEC," *Sensors*, vol. 23, no. 5, Art. no. 2565, 2023.

[14] H. He, X. Yang, X. Mi, H. Shen, and X. Liao, "Multi-agent deep reinforcement learning based dynamic task offloading in a device-to-device mobile-edge computing network to minimize average task delay with deadline constraints," *Sensors*, vol. 24, no. 16, Art. no. 5141, 2024.

[15] Z. Zhao, J. Perazzone, G. Verma, and S. Segarra, "Congestion-aware distributed task offloading in wireless multi-hop networks using graph neural networks," in *Proc. IEEE ICASSP*, 2024, pp. 8951–8955.

[16] J. Spenger, "Programming models for failure-transparent distributed systems," Ph.D. dissertation, KTH Royal Institute of Technology, Stockholm, Sweden, 2025.

[17] Termux, "Termux: Android terminal emulator and Linux environment." [Online]. Available: https://termux.dev

[18] PRoot, "chroot, mount –bind, and binfmt_misc without privilege/setup for Linux." [Online]. Available: https://proot-me.github.io

[19] D. Anderson, "How NAT traversal works," Tailscale blog, Aug. 2020. [Online]. Available: https://tailscale.com/blog/how-nat-traversal-works

[20] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *J. Cryptogr. Eng.*, vol. 2, no. 2, pp. 77–89, 2012.

[21] E. Rescorla, "Keying material exporters for Transport Layer Security (TLS)," RFC 5705, IETF, Mar. 2010.

[22] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, IETF, Feb. 1997.

[23] E. Rescorla, "The Transport Layer Security (TLS) protocol version 1.3," RFC 8446, IETF, Aug. 2018.

[24] S. Li et al., "PyTorch distributed: Experiences on accelerating data parallel training," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3005–3018, 2020.

[25] cloudpickle, "Extended pickling support for Python objects." [Online]. Available: https://github.com/cloudpipe/cloudpickle

[26] T. Dettmers, "8-bit approximations for parallelism in deep learning," in *Proc. ICLR*, 2016.

[27] F. Seide, H. Fu, J. Droppo, G. Li, and D. Yu, "1-bit stochastic gradient descent and its application to data-parallel distributed training of speech DNNs," in *Proc. Interspeech*, 2014, pp. 1058–1062.

[28] D. G. Thaler and C. V. Ravishankar, "Using name-based mappings to increase hit rates," *IEEE/ACM Trans. Netw.*, vol. 6, no. 1, pp. 1–14, Feb. 1998.

[29] A. Douillard et al., "DiLoCo: Distributed low-communication training of language models," arXiv:2311.08105, 2023.

[30] T. Lin, S. U. Stich, K. K. Patel, and M. Jaggi, "Don't use large mini-batches, use local SGD," in *Proc. ICLR*, 2020.

[31] T. Vogels, S. P. Karimireddy, and M. Jaggi, "PowerSGD: Practical low-rank gradient compression for distributed optimization," in *Proc. NeurIPS*, 2019.

[32] P. Goyal et al., "Accurate, large minibatch SGD: Training ImageNet in 1 hour," arXiv:1706.02677, 2017.

[33] E. Hoffer, I. Hubara, and D. Soudry, "Train longer, generalize better: Closing the generalization gap in large batch training of neural networks," in *Proc. NeurIPS*, 2017, pp. 1729–1739.

[34] T. B. Johnson, P. Agrawal, H. Gu, and C. Guestrin, "AdaScale SGD: A user-friendly algorithm for distributed training," in *Proc. 37th ICML*, PMLR vol. 119, 2020, pp. 4911–4920.

[35] A. Sarfi, B. Thérien, J. Lidin, and E. Belilovsky, "Overcoming the communication-performance tradeoff in LLM pretraining," arXiv:2508.15706, 2025.

[36] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," in *Proc. EMNLP*, 2017, pp. 440–445.

[37] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. MLSys*, vol. 2, 2020, pp. 429–450.